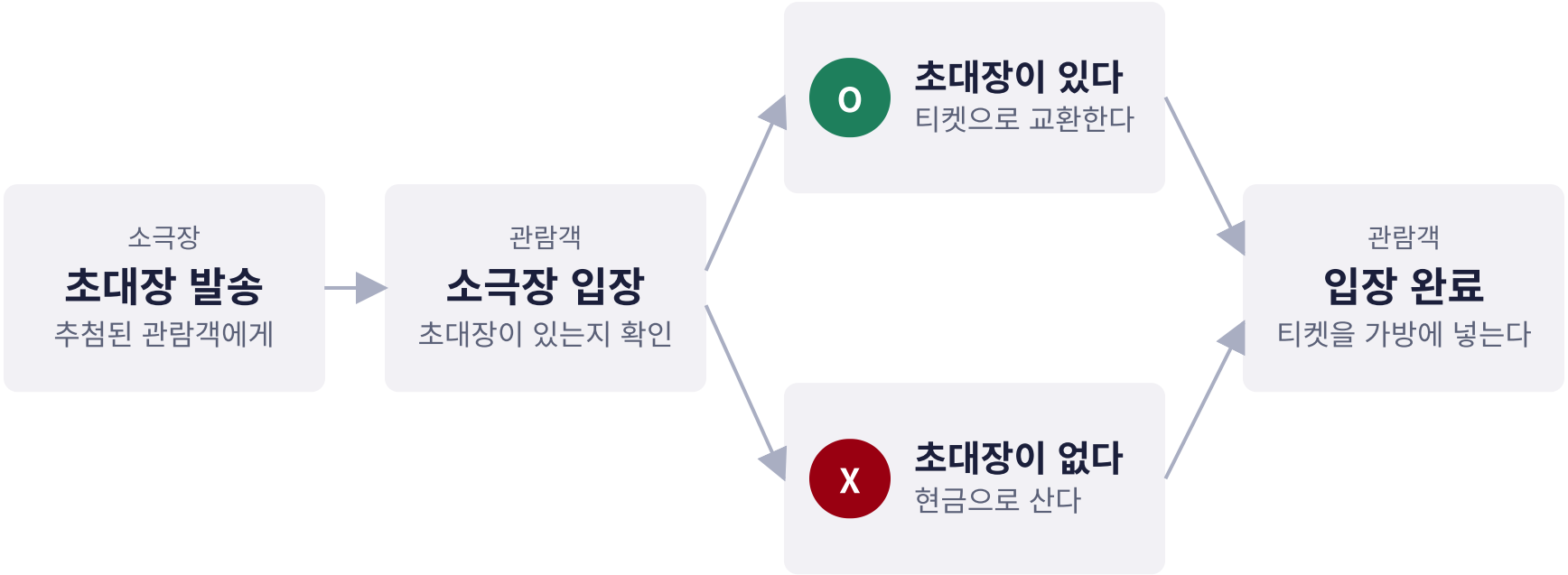
1장. 객체, 설계

티켓 판매 예제로 본 자율적인 객체

오브젝트 스터디 · Week 01

발표자: 이강욱

소극장 티켓 판매 예제


소극장
초대장 발송
추첨된 관람객에게
관람객
소극장 입장
초대장이 있는지 확인
O
초대장이 있다
티켓으로 교환한다
X
초대장이 없다
현금으로 산다
관람객
입장 완료
티켓을 가방에 넣는다

step01: 모든 것을 아는 Theater

step01 / Theater.kt

```
fun enter(audience: Audience) {
  if (audience.bag.hasInvitation()) {
    val ticket = ticketSeller.ticketOffice.getTicket()
    audience.bag.ticket = ticket
  } else {
    val ticket = ticketSeller.ticketOffice.getTicket()
    audience.bag.minusAmount(ticket.fee)
    ticketSeller.ticketOffice.plusAmount(ticket.fee)
    audience.bag.ticket = ticket
  }
}
```

1 **예상을 벗어난다**
소극장이 관람객 가방을 연다

2 **기억할 게 많다**
접근 대상의 내부 구조가 얽힌다

3 **바꾸기 어렵다**
내부 객체가 바뀌면 함께 바뀐다

빨간 글자: Theater가 다른 객체의 내부에 직접 손대는 곳

변경이 Theater로 번진다

1

지갑만 들고 온 관람객

audience.bag이
wallet으로 바뀐다

→ Theater 수정

2

키오스크를 쓰는 판매원

ticketSeller.ticketOffice가
바뀐다

→ Theater 수정

3

카드 결제 추가

결제 로직이
Theater 안에 있다

→ Theater 수정

의존성 한 객체가 바뀔 때, 그 객체에 의존하는 객체도 함께 바뀔 수 있는 성질
결합도 객체 사이 의존성의 정도. 의존성이 지나치면 결합도가 높다
→ 결합도가 높을수록 작은 변경에도 고칠 곳이 많아져, 바꾸기 어려운 코드가 된다

개선 전략은 하나

다른 객체의 내부를 뒤지던 코드를, 그 내부를 가진 객체 안으로 옮긴다

step 02 — TicketSeller
`sellTo()`
Theater가 매표소를 모른다

step 03 — Audience
`buy()`
판매원이 가방을 열지 않는다

step 04 — Bag
`payFor()`
관람객이 가방 속을 뒤지지 않는다

step 05 — TicketOffice
`sellTicketTo()`
판매원이 매표소 속을 뒤지지 않는다

트레이드오프

Theater가 알아야 할 것: 7개에서 1개로

step01 · Theater가 아는 것

7

- `audience.bag`
- `bag.hasInvitation()`
- `bag.ticket`
- `bag.minusAmount()`
- `ticketSeller.ticketOffice`
- `ticketOffice.getTicket()`
- `ticketOffice.plusAmount()`

step02 이후 · Theater가 아는 것

1

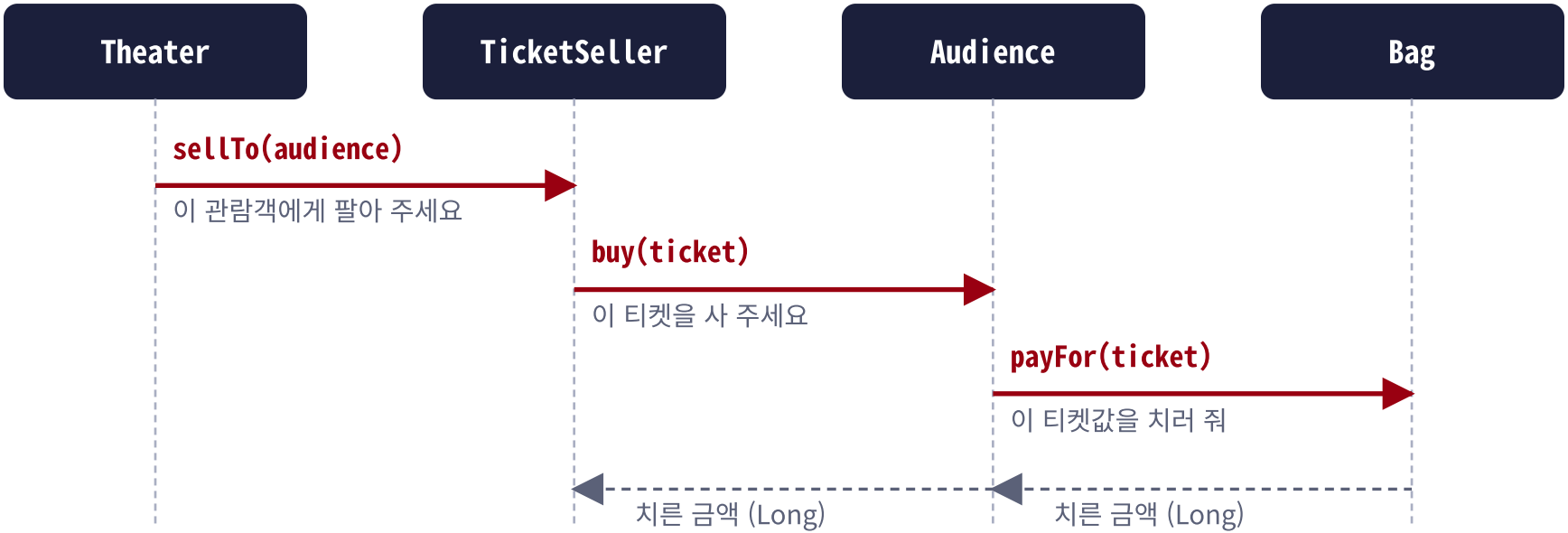
`ticketSeller.sellTo()`

TicketOffice가 있는지조차 모른다.
어떻게 팔지는 판매원이 정한다.

```
fun enter(audience: Audience) {
  ticketSeller.sellTo(audience)
}
```

인터페이스만 공개하고, 구현은 숨긴다

객체는 메시지로 협력한다



메시지는 명령이 아니라 요청이다 — 어떻게 처리할지는 받는 쪽이 정한다

개선에서 배운 개념

ENCAPSULATION

캡슐화와 인터페이스

- 내부는 private으로 숨기고 메시지만 공개
- 구현 변경은 안전, 인터페이스 변경은 전파

COHESION · COUPLING

응집도와 결합도

- 연관된 일만 하고 나머지는 맡긴다
- 자율적인 객체 → 결합도 ↓ 응집도 ↑

PROCEDURAL vs OBJECT-ORIENTED

절차지향 vs 객체지향

- 절차: 데이터와 처리를 따로
- 객체: 데이터와 처리를 함께

RESPONSIBILITY

책임의 이동

- Theater에 몰린 책임을 각 객체로
- 어떤 데이터보다 어떤 책임인가

반박 ①: hold 대신 payFor

책 **`hold(ticket)`** 보관하다

- O **티켓을 넣는다** 이름에 있음
- X **초대장이 없으면 돈을 뺀다** 이름에 없음
- X **뺀 금액을 반환한다** 이름에 없음

내 선택

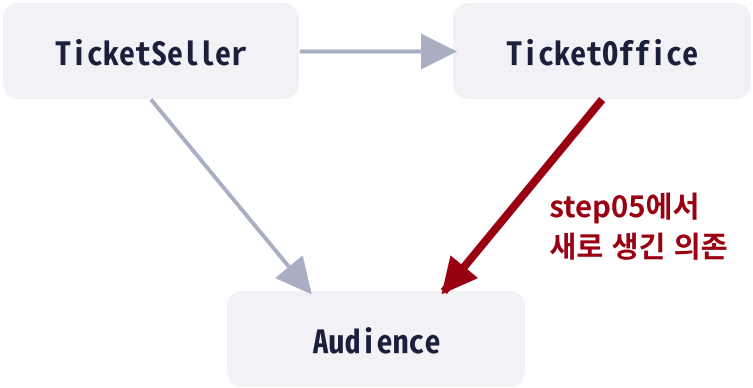
`payFor(ticket)`

이 티켓값을 치르고, 치른 금액을 알려줘
초대장이 있으면 → 0원을 치른 것

상태를 바꾸면서(명령) 값도 돌려준다(조회)
→ 명령-조회 분리(CQS) 원칙과 어긋난다

남은 아쉬움: '티켓을 넣는다'가 이름에서 빠진다
→ 이름에는 가장 중요한 책임 하나만 담는다

반박 ②: sellTicketTo의 트레이드오프



회색: 기존 의존　빨강: 새 의존

구분	step04 (판매원이 판매)	step05 (매표소가 판매)
TicketOffice 자율성	낮다	**높다**
Audience에 의존하는 객체	TicketSeller	**TicketSeller TicketOffice**

모든 객체를 자율적으로 만드는 것이 항상 정답은 아니다. **설계는 트레이드오프의 산물이다.**

좋은 설계는 의존성을 관리하는 설계

정리

1. **각 객체가 자기 문제를 스스로 해결한다**
   자율성
2. **내부는 숨기고 메시지로만 협력한다**
   캡슐화
3. **자율성과 결합도 사이에서 균형을 잡는다**
   트레이드오프

토론거리

Q1. TicketOffice에게도 자율성을 줘야 할까요?

매표소의 자율성 vs Audience에 대한 새 의존

step04 vs step05

Q2. hold와 payFor, 어느 이름이 더 나을까요?

이름에 담을 책임은 무엇인가